

DOCUMENTO CONSIDERADO CONFIDENCIAL E DE USO INTERNO

Configuração e deploy do sistema ZEHUA SmartHotel. Cobrindo ambientes locais, Docker, produção, integrações PMS e monitoramento.

SmartHotel

V1.0

Sumario

1. Pre-requisitos	3
1.1 Requisitos de Software	3
1.2 Requisitos de Hardware	3
1.3 Dominio e Certificado SSL	4
2. Instalacao Local	4
2.1 Clonar o Projeto	4
2.2 Instalar Dependencias	4
2.3 Configurar Variaveis de Ambiente	4
2.4 Inicializar o Banco de Dados	5
2.5 Iniciar o Servidor de Desenvolvimento	5
2.6 Resolucao de Problemas Comuns	5
3. Configuracao de Ambiente	5
3.1 Variaveis Essenciais	6
3.2 Chaves de Integracao	6
3.3 Configuracao SMTP	6
3.4 WhatsApp Evolution API	7
4. Build de Producao	7
4.1 Gerar Build	7
4.2 Executar em Producao	7
4.3 Gerenciamento com PM2	7
5. Deploy com Docker	8
5.1 Arquitetura de Containers	8
5.2 Configuracao do Docker Compose	8
5.3 Volumes e Persistencia	9
5.4 Health Checks	9
6. Deploy com Docker Compose Producao	9

6.1 Multi-Stage Build	9
6.2 Caddy como Reverse Proxy	9
6.3 Comandos de Deploy	10
7. Integracao com PMS	10
7.1 Mews Systems	10
7.2 Cloudbeds	10
7.3 Oracle OHIP	11
7.4 Teste de Integracoes	11
8. Configuracao de Dominio e SSL	11
8.1 Configuracao DNS	11
8.2 SSL Automatico com Caddy	12
8.3 Cloudflare (Opcional)	12
9. Monitoramento e Observabilidade	12
9.1 Pilares de Observabilidade	12
9.2 Endpoints de Health Check	12
9.3 Configuracao de Alertas	13
10. Backup e Recovery	13
10.1 Backup do PostgreSQL	13
10.2 Backup do Redis	13
10.3 Procedimento de Recovery	13
11. Escalonamento	14
11.1 Escalonamento Horizontal	14
11.2 Redis como Store de Sessao	14
11.3 Connection Pooling e CDN	14

1. Pre-requisitos

Antes de iniciar a instalacao do ZEHLA SmartHotel Cognitive Hospitality OS, e necessario garantir que o ambiente atenda aos requisitos minimos de software e hardware. Este guia assume familiaridade basica com linha de comando Linux e conceitos de servidores web.

1.1 Requisitos de Software

O sistema e construido sobre uma stack moderna de desenvolvimento. A tabela abaixo lista todos os componentes necessarios e suas versoes recomendadas:

Componente	Versao Minima	Recomendada	Observacao
Node.js	18.x LTS	20.x LTS	Runtime principal do Next.js
npm	9.x	10.x	Gerenciador de pacotes
PostgreSQL	14.x	16.x	Banco de dados relacional principal
Supabase	-	Ultima	Alternativa gerenciada ao PostgreSQL
Redis	7.x	7.2+	Cache de sessao, filas e rate limiting
Docker	24.x	25.x	Opcional - para deploy containerizado
Docker Compose	2.20+	2.24+	Orquestracao de containers
Git	2.40+	2.43+	Controle de versao
PM2	5.x	5.3+	Gerenciador de processos para producao

Tabela 1: Requisitos de software e versoes

1.2 Requisitos de Hardware

O dimensionamento minimo de hardware depende do volume esperado de hospedes e transacoes. Para deployments iniciais e ambientes de teste, as especificacoes abaixo sao suficientes:

Recurso	Deploy Pequeno	Deploy Medio	Deploy Grande
vCPU	2 cores	4 cores	8+ cores
RAM	4 GB	8 GB	16+ GB
Armazenamento (SSD)	20 GB	50 GB	100+ GB
Largura de banda	10 Mbps	50 Mbps	100+ Mbps

Uso típico	Ate 50 quartos	50-200 quartos	200+ quartos
------------	----------------	----------------	--------------

Tabela 2: Dimensionamento de hardware por porte

1.3 Dominio e Certificado SSL

Um dominio registrado e obrigatorio para producao. O certificado SSL/TLS e obtido automaticamente via Let's Encrypt quando utilizando Caddy como reverse proxy, ou pode ser provisionado manualmente via Certbot para Nginx/Apache. Recomenda-se forcar redirecionamento HTTPS em todo o trafego.

2. Instalacao Local

A instalacao local permite desenvolver e testar o ZEHLA SmartHotel em seu workstation antes de realizar o deploy em ambiente de producao. Siga os passos abaixo para configurar o ambiente de desenvolvimento completo.

2.1 Clonar o Projeto

Obtenha o codigo-fonte atraves de Git ou por download do arquivo compactado. Se utilizar Git, autentique-se com suas credenciais de acesso ao repositorio privado:

```
git clone https://github.com/zehla-hospitality/zehla-os.git
cd zehla-os
```

2.2 Instalar Dependencias

Execute o comando de instalacao do npm para baixar todas as dependencias listadas no arquivo package.json. Este processo pode levar alguns minutos dependendo da velocidade da conexao:

```
npm install
```

2.3 Configurar Variaveis de Ambiente

Copie o arquivo de exemplo e ajuste as variaveis conforme seu ambiente local. Este passo e critico para o funcionamento correto do sistema:

```
cp .env.example .env
```

Edite o arquivo .env com seu editor de preferencia e configure as variaveis essenciais: DATABASE_URL (conexao com PostgreSQL), NEXTAUTH_SECRET (chave de seguranca para autenticacao), NEXTAUTH_URL (URL base da aplicacao) e REDIS_URL (conexao com Redis). Detalhes completos na

secao 3.

2.4 Inicializar o Banco de Dados

O ZEHLA utiliza Prisma ORM para gerenciamento do schema do banco. Execute o comando de push para criar todas as tabelas necessarias no PostgreSQL:

```
npx prisma db push
```

Caso prefira utilizar migracoes versionadas (recomendado para producao), use `npx prisma migrate dev` durante o desenvolvimento.

2.5 Iniciar o Servidor de Desenvolvimento

Com tudo configurado, inicie o servidor de desenvolvimento:

```
npm run dev
```

Acesse `http://localhost:3000` no navegador para visualizar a aplicacao. O servidor suporta hot-reload, ou seja, alteracoes no codigo sao refletidas automaticamente sem necessidade de reinicio.

2.6 Resolucao de Problemas Comuns

Abaixo estao os problemas mais frequentes encontrados durante a instalacao local e suas solucoes:

Problema	Causa Provavel	Solucao
Erro ECONNREFUSED no PostgreSQL	PostgreSQL nao esta rodando	<code>systemctl start postgresql</code>
Erro de autenticao Prisma	DATABASE_URL incorreta	Verifique usuario, senha e host no <code>.env</code>
Porta 3000 em uso	Outro processo na porta	<code>npm run dev -- -p 3001</code>
Modulo nao encontrado	node_modules incompleto	<code>rm -rf node_modules && npm install</code>
Redis connection refused	Redis nao iniciado	redis-server ou Docker: <code>docker run redis</code>
Erro de permissao no Linux	Diretorio sem permissao	<code>sudo chown -R \$USER:\$USER ./</code>

Tabela 3: Problemas comuns e solucoes

3. Configuracao de Ambiente

O arquivo .env é o ponto central de configuração do ZEHLA SmartHotel. Todas as integrações, credenciais e parâmetros operacionais são definidos neste arquivo. Nunca exponha o arquivo .env em repositórios públicos.

3.1 Variáveis Essenciais

As variáveis a seguir são obrigatórias para o funcionamento básico do sistema:

Variável	Descrição	Exemplo
DATABASE_URL	String de conexão PostgreSQL	postgresql://user:pass@localhost:5432/zehla
NEXTAUTH_URL	URL base da aplicação	https://app.seudominio.com
NEXTAUTH_SECRET	Chave criptográfica (min 32 chars)	gerado com openssl rand -base64 32
REDIS_URL	String de conexão Redis	redis://localhost:6379
NODE_ENV	Ambiente de execução	development production

Tabela 4: Variáveis essenciais de ambiente

3.2 Chaves de Integração

O ZEHLA integra-se com diversos serviços externos de inteligência artificial e pagamentos. Configure as chaves de API conforme necessário para cada módulo ativo:

Variável	Serviço	Onde Obter
ZAI_API_KEY	API de IA cognitiva Z.ai	Portal Z.ai Developer
PAGARME_API_KEY	Gateway de pagamentos Pagarme	dashboard.pagarme.com.br
MEWS_API_KEY	PMS Mews Systems	app.mews.com/api
CLOUDBEDS_API_KEY	PMS Cloudbeds	hotels.cloudbeds.com/api
STRIPE_SECRET_KEY	Pagamentos internacionais	dashboard.stripe.com/apikeys

Tabela 5: Chaves de API para integrações

3.3 Configuração SMTP

Para envio de e-mails transacionais (confirmações de reserva, notificações, recuperação de senha), configure os parâmetros SMTP. Recomenda-se utilizar um serviço dedicado como Amazon SES, SendGrid ou Mailgun para alta deliverabilidade:

```
SMTP_HOST=smtp.sendgrid.net
SMTP_PORT=587
SMTP_USER=apikey
SMTP_PASSWORD=sua_chave_sendgrid
SMTP_FROM=noreply@seudominio.com
SMTP_SECURE=true
```

3.4 WhatsApp Evolution API

O modulo de comunicacao via WhatsApp utiliza a Evolution API para envio e recebimento de mensagens. Configure a URL da instancia e a chave de autenticacao no arquivo .env. A API suporta envio de mensagens de texto, midia, templates e listas interativas.

4. Build de Producao

O build de producao otimiza a aplicacao para performance maxima, gerando arquivos estaticos, minificacao de JavaScript/CSS e pre-compilacao de paginas. Este processo deve ser executado em ambiente de staging antes de qualquer deploy.

4.1 Gerar Build

Execute o comando de build para compilar a aplicacao. O Next.js gera uma saida standalone que inclui apenas os arquivos necessarios para execucao:

```
npm run build
```

A saida e gerada no diretorio .next/standalone. Certifique-se de que nao houve erros durante o processo. O output standalone e utilizado pelo Dockerfile de producao.

4.2 Executar em Producao

Apos o build, inicie o servidor de producao que roda na porta 3000 por padrao:

```
npm start
```

O servidor de producao oferece desempenho superior ao modo de desenvolvimento, sem hot-reload e com otimizacoes de compilacao ativadas.

4.3 Gerenciamento com PM2

Para garantir alta disponibilidade, utilize o PM2 como gerenciador de processos. Ele oferece reinicio automatico em caso de falhas, monitoramento de recursos e gerenciamento de logs:

```
# Instalacao global do PM2
npm install -g pm2

# Iniciar aplicacao com PM2
pm2 start npm --name "zehla" -- start

# Salvar configuracao para reinicio automatico
pm2 save
pm2 startup
```

Comandos uteis do PM2: pm2 logs zehla (visualizar logs), pm2 restart zehla (reiniciar), pm2 monit (monitoramento em tempo real), pm2 stop zehla (parar). Configure rotacao de logs com pm2 install pm2-logrotate para evitar crescimento indefinido dos arquivos de log.

5. Deploy com Docker

O Docker oferece um ambiente isolado e reproduzível, eliminando problemas de compatibilidade entre ambientes. O ZEHLA fornece um docker-compose.yml padrao que orquestra todos os servicos necessarios.

5.1 Arquitetura de Containers

O docker-compose.yml define tres servicos principais que trabalham em conjunto:

Servico	Imagem/Base	Porta	Funcao
app	Node.js 20 Alpine	3000	Aplicacao ZEHLA SmartHotel
postgres	PostgreSQL 16 Alpine	5432	Banco de dados relacional
redis	Redis 7 Alpine	6379	Cache, sessoes e filas

Tabela 6: Servicos Docker do ZEHLA

5.2 Configuracao do Docker Compose

Antes de iniciar, copie o arquivo de configuracao de ambiente e ajuste as variaveis:

```
cp .env.example .env
# Edite o .env com as configuracoes adequadas

# Iniciar todos os servicos
```

```
docker compose up -d

# Verificar status dos containers
docker compose ps

# Visualizar logs
docker compose logs -f app
```

5.3 Volumes e Persistencia

Os dados do PostgreSQL e Redis sao persistidos via Docker volumes nomeados. Isso garante que os dados sobrevivam a reinicializacao e recriacao dos containers. O volume pgdata armazena os dados do banco e o volume redisdata armazena os snapshots do Redis. Diretorios de upload e arquivos estaticos tambem sao montados como bind mounts para facilitar o acesso.

5.4 Health Checks

Todos os containers incluem configuracoes de health check que monitoram a disponibilidade dos servicos. O container da aplicacao expoe o endpoint /api/health que retorna status 200 quando o sistema esta operacional. O Docker Compose utiliza estes checks para decidir sobre restart automatico e dependencia de inicializacao entre servicos.

6. Deploy com Docker Compose Producao

Para ambientes de producao, o ZEHLA fornece um docker-compose.prod.yml otimizado com multi-stage build, reverse proxy Caddy, SSL/TLS automatico via Let's Encrypt e compressao gzip.

6.1 Multi-Stage Build

O Dockerfile de producao utiliza build em multiplas etapas. A primeira etapa (builder) instala dependencias e realiza o build da aplicacao. A segunda etapa (runner) copia apenas os artefatos necessarios para a imagem final, resultando em um tamanho significativamente menor e maior seguranca.

6.2 Caddy como Reverse Proxy

O Caddy e utilizado como reverse proxy em producao devido a sua configuracao simplificada e provisionamento automatico de certificados SSL/TLS via Let's Encrypt. Ele tambem fornece compressao gzip automatica, HTTP/2 e HTTP/3 nativos, e redirecionamento HTTPS forçado.

```
# Caddyfile de exemplo
seudominio.com {
  reverse_proxy app:3000
  encode gzip
}
```

6.3 Comandos de Deploy

Para iniciar o ambiente de producao completo:

```
# Build e inicio em modo producao
docker compose -f docker-compose.prod.yml up -d --build

# Verificar logs de todos os servicos
docker compose -f docker-compose.prod.yml logs -f

# Reiniciar servico especifico
docker compose -f docker-compose.prod.yml restart app
```

7. Integracao com PMS

O ZEHLA SmartHotel suporta integracao com os principais Property Management Systems (PMS) do mercado. Estas integracoes permitem sincronizacao bidirecional de reservas, disponibilidade, precos e dados de hospedes em tempo real.

7.1 Mews Systems

O Mews e um dos PMS mais utilizados em hotéis boutique e redes independentes. A integracao utiliza a API REST do Mews para sincronizacao de reservas e ocupacao:

1. Acesse o painel do Mews e navegue ate Settings > Api > Api tokens
2. Crie um novo token de API com permissoes de leitura e escrita para reservas
3. Copie a chave de API (MEWS_API_KEY) e configure no .env do ZEHLA
4. Configure o Webhook URL apontando para <https://seudominio.com/api/webhooks/mews>
5. Realize o mapeamento de propriedades no painel de configuracao do ZEHLA
6. Execute um teste de sincronizacao via interface administrativa do ZEHLA

7.2 Cloudbeds

O Cloudbeds oferece uma plataforma completa de gestao hoteleira. A integracao sincroniza reservas, disponibilidade e dados de hospedes:

1. Obtenha as credenciais de API no painel do Cloudbeds (Developer Portal)
2. Configure CLOUDBEDS_API_KEY e CLOUDBEDS_PROPERTY_ID no .env
3. Defina o intervalo de sincronizacao recomendado: 15 minutos para reservas
4. Ative webhooks para eventos de nova reserva, cancelamento e modificacao
5. Mapeie os tipos de quarto entre Cloudbeds e ZEHLA no painel de administracao

7.3 Oracle OHIP

Para grandes redes hoteleiras que utilizam Oracle Hospitality Integration Platform (OHIP), o ZEHLA suporta conectividade via endpoints OHIP. A configuracao requer credenciais de acesso ao Oracle Cloud e configuracao de endpoint especifico da propriedade. Consulte a documentacao especifica do OHIP para detalhes de autenticao OAuth2.

7.4 Teste de Integracoes

Apos configurar qualquer PMS, utilize o painel de diagnostico integrado do ZEHLA para verificar a conectividade. O endpoint `/api/admin/integrations/test` realiza testes de conexao, autenticao e sincronizacao de dados de exemplo. Registre os resultados dos testes para auditoria.

8. Configuracao de Dominio e SSL

A configuracao correta de dominio e SSL e essencial para a seguranca e confiabilidade da aplicacao em producao. Esta secao detalha os passos necessarios.

8.1 Configuracao DNS

No painel de gerenciamento do seu provedor de dominio (Registro.br, Cloudflare, Route 53), crie os seguintes registros DNS apontando para o endereco IP do servidor de producao:

Tipo	Nome	Valor	TTL
A	@	IP_DO_SERVIDOR	300
A	app	IP_DO_SERVIDOR	300
CNAME	www	seudominio.com	3600
TXT	@	v=spf1 include:... ~all	3600

Tabela 7: Registros DNS necessarios

8.2 SSL Automatico com Caddy

O Caddy provisiona automaticamente certificados SSL/TLS via ACME (Let's Encrypt). Basta configurar o dominio no Caddyfile e o certificado sera obtido e renovado automaticamente. O processo e totalmente transparente e nao requer intervencao manual para renovacoes.

8.3 Cloudflare (Opcional)

Para protecao adicional contra DDoS e melhoria de performance via CDN, configure o Cloudflare como proxy DNS. Altere os nameservers do dominio para os do Cloudflare e ative o proxy (nuvem laranja) para o registro A. Configure o modo SSL como "Full (Strict)" no painel do Cloudflare para garantir comunicacao criptografada ponta-a-ponta.

9. Monitoramento e Observabilidade

O monitoramento eficaz e fundamental para manter a disponibilidade e performance do ZEHLA SmartHotel. O sistema integra-se com ferramentas de observabilidade modernas para rastreamento distribuido, agregacao de logs e alertas.

9.1 Pilares de Observabilidade

A arquitetura de observabilidade do ZEHLA e baseada em tres pilares fundamentais:

Pilar	Ferramenta	Funcao
Tracing	OpenTelemetry + Tempo	Rastreamento distribuido de requisicoes
Métricas	OpenTelemetry + Prometheus	Coleta e armazenamento de metricas de performance
Logs	OpenTelemetry + Loki	Agregacao centralizada de logs de aplicacao
Erros	Sentry	Rastreamento de erros e excecoes em tempo real

Tabela 8: Pilares de observabilidade

9.2 Endpoints de Health Check

O sistema expoe endpoints dedicados para verificacao de saude de cada componente. O endpoint principal /api/health retorna o status de todos os servicos dependentes (banco de dados, Redis, APIs externas). Monitore estes endpoints com ferramentas como Uptime Robot, Better Uptime ou Grafana

Alerting.

9.3 Configuracao de Alertas

Configure alertas no Sentry para captura automatica de erros nao tratados. No Loki, configure regras de alerta para padroes de log criticos (erros 5xx repetidos, falhas de autenticacao em massa). Integre com canais de notificacao como Slack, PagerDuty ou e-mail para garantir resposta rapida a incidentes.

10. Backup e Recovery

Uma estrategia robusta de backup e essencial para protecao contra perda de dados. O ZEHLA suporta backup automatizado de banco de dados, cache e sistema de arquivos.

10.1 Backup do PostgreSQL

Utilize `pg_dump` para backup completo do banco de dados. Recomenda-se agendar backups diarios via cron e manter pelo menos 7 dias de retencao. Para ambientes com alto volume, considere backups incrementais com `pgBackRest`:

```
# Backup completo
pg_dump -U zehla_user -F c -f /backups/zehla_$(date +%Y%m%d).dump zehla_db

# Cron job diario (02:00)
0 2 * * * pg_dump -U zehla_user -F c -f /backups/zehla_$(date +%Y%m%d).dump
zehla_db
```

10.2 Backup do Redis

O Redis suporta snapshots automaticos (RDB) configurados via diretiva `save` no arquivo `redis.conf`. Recomenda-se salvar a cada 15 minutos se houver ao menos 1000 alteracoes. Backup dos snapshots: copie periodicamente o arquivo `dump.rdb` para um local de armazenamento externo.

10.3 Procedimento de Recovery

Para restaurar o banco de dados a partir de um backup, utilize `pg_restore`. Em caso de desastre total, siga o plano de disaster recovery: (1) restaure o backup mais recente do PostgreSQL, (2) restaure o snapshot do Redis, (3) recrie a infraestrutura de containers via docker compose, (4) valide integridade dos dados e (5) retome operacoes normais.

11. Escalonamento

A arquitetura do ZEHLA foi projetada para escalar horizontalmente conforme a demanda cresce. Esta seção descreve as estratégias de escalonamento para diferentes níveis de carga.

11.1 Escalonamento Horizontal

Para lidar com aumento de tráfego, implante múltiplas instâncias da aplicação atrás de um load balancer. O Docker Compose suporta escala via comando `docker compose up -d --scale app=N`. Para produção, utilize um orquestrador como Kubernetes ou um serviço gerenciado como AWS ECS, Google Cloud Run ou Railway.

11.2 Redis como Store de Sessão

Com múltiplas instâncias da aplicação, o Redis atua como store centralizado de sessão, garantindo que qualquer instância possa processar requisições de qualquer usuário sem perda de contexto. Configure `NEXTAUTH_URL` com o domínio compartilhado e `NEXTAUTH_SECRET` idêntico em todas as instâncias.

11.3 Connection Pooling e CDN

Configure o connection pooling do PostgreSQL via PgBouncer para gerenciar eficientemente as conexões com o banco em cenários de múltiplas instâncias. Para assets estáticos, utilize uma CDN como Cloudflare ou AWS CloudFront para reduzir a carga no servidor de origem e melhorar o tempo de carregamento para usuários geograficamente distantes.